INSTITUTO FEDERAL DO PIAUÍ
CAMPUS FLORIANO
CURSO: Técnico Integrado em Informática
DISCIPLINA: Programação Orientada a Objetos
PROFESSOR: Ronaldo Pires Borges

Exercício 8: Tratamento de Exceções em Java

Nível Básico: Lançamento e Captura Simples

1. Lançando uma Exceção Básica (throw new)

Crie uma classe chamada ValidadorIdade com um método estático verificarIdade(int idade). Se a idade informada for menor que 18 anos, o método deve lançar explicitamente uma exceção do tipo IllegalArgumentException com a mensagem "Idade não permitida: menor de idade". Caso contrário, exiba "Acesso liberado". Escreva uma classe de teste (TestaIdade) com um bloco try-catch para capturar a exceção e imprimir a mensagem de erro.

2. Capturando Exceções de Divisão por Zero (ArithmeticException)

Escreva um programa em Java que solicite ao usuário dois números inteiros (a e b) via teclado (usando a classe Scanner). O programa deve realizar a divisão de a por b. Utilize o bloco try-catch para capturar especificamente a exceção ArithmeticException que ocorre em caso de divisão por zero, exibindo uma mensagem amigável ao usuário (ex: "Erro: Não é possível dividir por zero!").

3. Tratamento de Conversão de Tipos (NumberFormatException)

Crie um método que receba uma String representando um número e tente convertê-la para um número inteiro utilizando Integer.parseInt(). Caso o usuário passe uma string inválida (ex: "abc"), o método deve capturar a exceção NumberFormatException e retornar o valor padrão 0, além de avisar que a conversão falhou.

Nível Intermediário: Múltiplos Catch e Bloco finally

4. Múltiplos Blocos catch

Crie um programa que receba um array de 3 posições de inteiros pré-preenchido. O programa deve tentar acessar um índice informado pelo usuário e dividir o número armazenado nessa posição por outro número também fornecido pelo usuário. Utilize múltiplos blocos catch para tratar separadamente:

- ArrayIndexOutOfBoundsException (caso o índice não exista no array).
- ArithmeticException (caso ocorra divisão por zero).
- Uma mensagem genérica para qualquer outra exceção imprevista.

5. O Uso Obrigatório do Bloco finally

Simule a leitura de um arquivo ou a abertura de uma conexão fictícia utilizando um bloco try-catch-finally. No bloco try, simule o lançamento de uma exceção (ex: RuntimeException). No bloco catch, trate a exceção exibindo sua mensagem. Garanta que o bloco finally seja sempre executado, imprimindo uma mensagem que indique que a conexão ou recurso foi encerrado/liberado com sucesso, independentemente de ter ocorrido erro ou não.

6. Hierarquia de Exceções e Ordem dos catch

Analise o código abaixo e responda justificando: o que acontece se colocarmos um bloco catch (Exception e) antes de um bloco catch (ArithmeticException e)? Explique por que a ordem das exceções importa no Java (conceito de classes genéricas vs. específicas na árvore de exceções).

```
try {
    int resultado = 10 / 0;
} catch (Exception e) {
    System.out.println("Erro genérico");
} catch (ArithmeticException e) {
    System.out.println("Erro de divisão por zero");
}
```

Nível Avançado: Exceções Personalizadas e Propagação (throws)

7. Criando uma Exceção Personalizada Não Checada (RuntimeException)

Crie uma exceção personalizada chamada SaldoInsuficienteException que herde de RuntimeException. Em seguida, crie uma classe ContaBancaria com atributos saldo e um método sacar(double valor). Se o valor do saque for superior ao saldo disponível, lance a sua exceção personalizada. Teste a aplicação tentando sacar um valor maior que o saldo.

8. Propagação de Exceções (throws)

Modifique a classe ContaBancaria da questão anterior para criar uma exceção personalizada checada (*checked exception*), fazendo-a herdar diretamente de Exception. Como consequência, os métodos que chamam sacar() agora são obrigados a tratar a exceção com try-catch ou propagá-la na assinatura do método utilizando a palavra-chave throws. Implemente a propagação até o método main.

9. Refatoração com Validação Centralizada e Evitação de if/else

Em sistemas orientados a objetos, validações repetidas de valores (como verificar se um valor é menor ou igual a zero em métodos de depósito e saque) poluem o código com condicionais if/else. Crie um método privado auxiliar chamado validarValor(double valor) que lance uma exceção caso o valor seja inválido. Refatore os métodos de uma classe Conta para utilizá-lo, removendo os blocos condicionais tradicionais de checagem.

10. Análise de Escopo e Fluxo de Execução com Exceções Aninhadas

Analise o trecho de código abaixo e determine exatamente qual será a saída impressa no console ao ser executado, justificando o fluxo de execução entre os blocos try, catch internos e externos:

```
try {
    System.out.println("1. Bloco try externo iniciado.");
    try {
        System.out.println("2. Bloco try interno iniciado.");
        throw new NullPointerException("Erro interno proposital");
    } catch (ArithmeticException e) {
        System.out.println("3. Catch interno (ArithmeticException)");
    }
    System.out.println("4. Linha interna pós-try");
} catch (NullPointerException e) {
    System.out.println("5. Catch externo capturou: " + e.getMessage());
} finally {
    System.out.println("6. Bloco finally externo executado.");
}
```